CHAPTER 4

SYSTEM DESIGN

4.1 High Level Design

The suggested framework is implemented as a modular pipeline that is supposed to analyze neuroimaging data and perform subject-level classification between Autism Spectrum Disorder (ASD) and typical control (TC). The framework usually employs functional MRI (fMRI) data from the ABIDE dataset and employs a graph-based deep learning paradigm to discover and learn differential patterns of functional connectivity. The framework is made up of five major components: data preparation, feature extraction, graph construction, classification, and visualization.

The protocol begins with data preparation, during which resting-state fMRI signals are parcellated with the Harvard-Oxford atlas in order to obtain region-wise time series. Pairwise Pearson correlation coefficients between all possible pairs of ROIs are computed in order to form a 110×110 symmetric functional connectivity matrix per subject. Matrices are then flattened in order to obtain high-dimensional feature vectors of brain connectivity.

Then, the feature extraction module employs Recursive Feature Elimination with linear SVM (RFE-SVM) to achieve dimensionality reduction while preserving the most informative features. This leaves a lower-dimensional feature space of 2,052 features per subject, with reduced noise and computational complexity for downstream processing.

During the graph construction phase, a subject graph is generated in which each node is an individual subject. Associations are formed based on feature similarity, computed via Euclidean distance, as well as phenotypic characteristics, including age, gender, and location of acquisition. The resultant graph structure encodes statistical as well as demographic interconnectivity among subjects.

The classification module uses a n-layer Graph Convolutional Network (GCN) to learn node embeddings, which are fed into a Multi-Layer Perceptron (MLP) to output the final prediction. Lastly, the visualization module visualizes an interactive 3D brain highlighting the most significant regions that contribute to each subject's classification, allowing end users like

clinicians or researchers to better understand.

4.1.1 Module Classification

The system can be split into the following modules, given in the table below:

Module	Description
Data Preparation	Processes ABIDE fMRI data, extracts ROI time series using the Harvard-Oxford atlas, and computes pairwise Pearson correlations to form connectivity matrices.
Feature Extraction	Applies Recursive Feature Elimination with SVM (RFE-SVM) to select the most informative functional connectivity features.
Graph Construction	Constructs a subject-level graph where nodes represent subjects and edges encode phenotypic and feature-based similarities.
GCN Classification	Uses PyTorch Geometric to run an n-layer GCN on the graph. Learned node embeddings are passed to an MLP for classification.
Visualization & Output	Displays classification results along with an interactive 3D brain visualization of important features contributing to prediction.

Table 4.1: Module classification for the System

The first step of the data preparation stage involves preprocessing the ABIDE data. The ABIDE dataset is preprocessed using different pipelines like NIAK and CPAC. Then it is possible to get the Harvard-Oxford atlas data that defines 110 regions of interest, whose BOLD signal values are defined over a time series. Pearson correlation is then applied to each subject's atlas data to get a 110 x 110 matrix defining the linear relationship between pairs of ROIs. They represent the functional connectivity between different regions.

RFE-SVM is used to extract top 2052 features which are used in the construction of node features in the graph. The system implements a node classification problem with each node denoting a particular subject.

The system also uses learned gradients to determine the important node features across all nodes or subjects. These are visualized in a graph.

4.1.2 System Architecture

The raw rs-fMRI data from the ABIDE dataset is first parcellated using the Harvard-Oxford (HO) atlas into 110 brain regions. For each subject, mean time series signals are extracted from these regions. Functional connectivity is then computed using Pearson correlation, generating a 110×110 symmetric matrix per subject. To reduce redundancy, the upper triangular portion (excluding the diagonal) is flattened into a 5,995-dimensional feature vector. To select the most informative features, Recursive Feature Elimination with Support Vector Machine (RFE-SVM) is applied, ranking and retaining the top 2,052 features. This process results in an 871×2,052 feature matrix used for graph construction and classification, capturing the most relevant connectivity patterns for distinguishing between ASD and TC subjects.

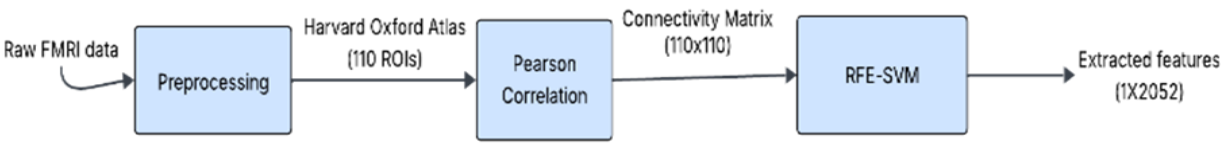

Fig 4.1: Feature extraction framework

Using the chosen 2,052-dimensional features, a population graph is built such that every node is a subject. Pairwise Euclidean similarity of the feature vectors defines edges between nodes, supplemented with phenotypic detail (age, gender, and site of acquisition) to build a complete adjacency matrix. The weighted graph is then used as input to a Graph Convolutional Network (GCN), which diffuses and sums over neighborhood information to learn strong node embeddings. A two-layer GCN is used to extract higher-order relations between subjects. The node representations resulting from it are fed into a Multi-Layer Perceptron (MLP) classifier with ReLU activation and softmax output, predicting whether each subject is in the ASD or control group. The end-to-end model combines feature and relational information for correct classification.

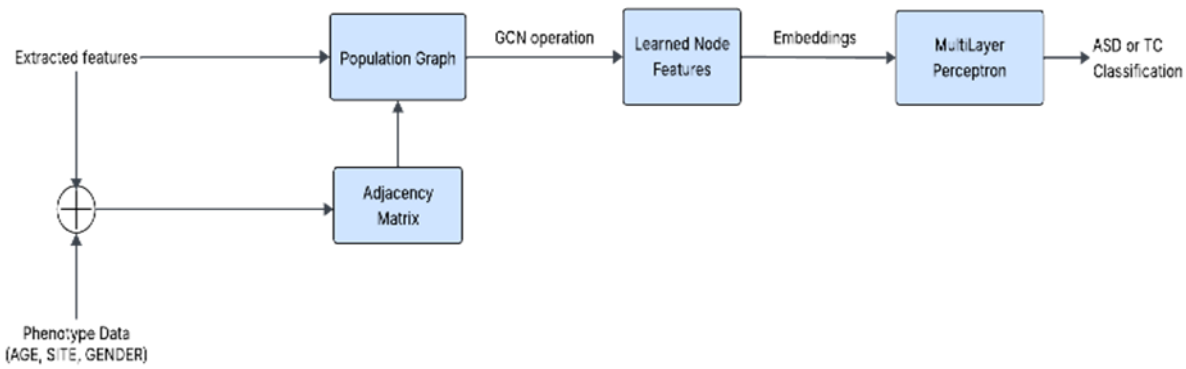

Fig 4.2: Graph construction and classification

The below table summarizes the summarizes the model architecture and hyperparameters used in the system:

Component	**Details**
Number of GCN Layers	7
Hidden Layer Dimension	128
K value	3
Learning Rate	0.005
Weight Decay	5e-4
Dropout Rate	0.2
Activation Function	ReLU
Output Layer	MLP with Softmax
Loss Function	Categorical Cross-Entropy

Table 4.2: Model architecture parameters

The ASD classification framework leverages a Graph Convolutional Network (GCN)-based architecture to model functional connectivity patterns derived from resting-state fMRI data. The overall model, built with 7 GCN layers, is designed to learn from the graph-structured representations of brain regions and their functional interactions.

Each GCN layer performs localized spectral filtering on the graph using Chebyshev polynomial approximations up to K=3, enabling efficient multi-hop message passing and neighborhood aggregation. The hidden layer dimensionality is set to 128, allowing the network to learn rich feature representations for each node (i.e., brain region).

The ReLU (Rectified Linear Unit) activation function is used after each GCN layer to introduce non-linearity, enabling the model to capture complex patterns in the brain connectivity data. To prevent overfitting, a dropout rate of 0.2 is applied during training, randomly deactivating a fraction of the neurons.

The output from the final GCN layer is passed to a Multi-Layer Perceptron (MLP) classifier, which uses a softmax activation in the output layer to generate class probabilities corresponding to diagnostic labels (e.g., ASD vs Control). The model is trained using categorical cross-entropy loss, which is suitable for multi-class classification problems.

To optimize the model, the Adam optimizer is used with a learning rate of 0.005 and weight decay of 5e-4, aiding in generalization and preventing overfitting by penalizing large weight values.

Overall, this architecture effectively captures both local and global topological properties of the brain graph, making it highly suitable for detecting subtle neurodevelopmental differences characteristic of ASD.

4.2 Detailed Design

4.2.1 Class Diagram

The class diagram for the system is given below:

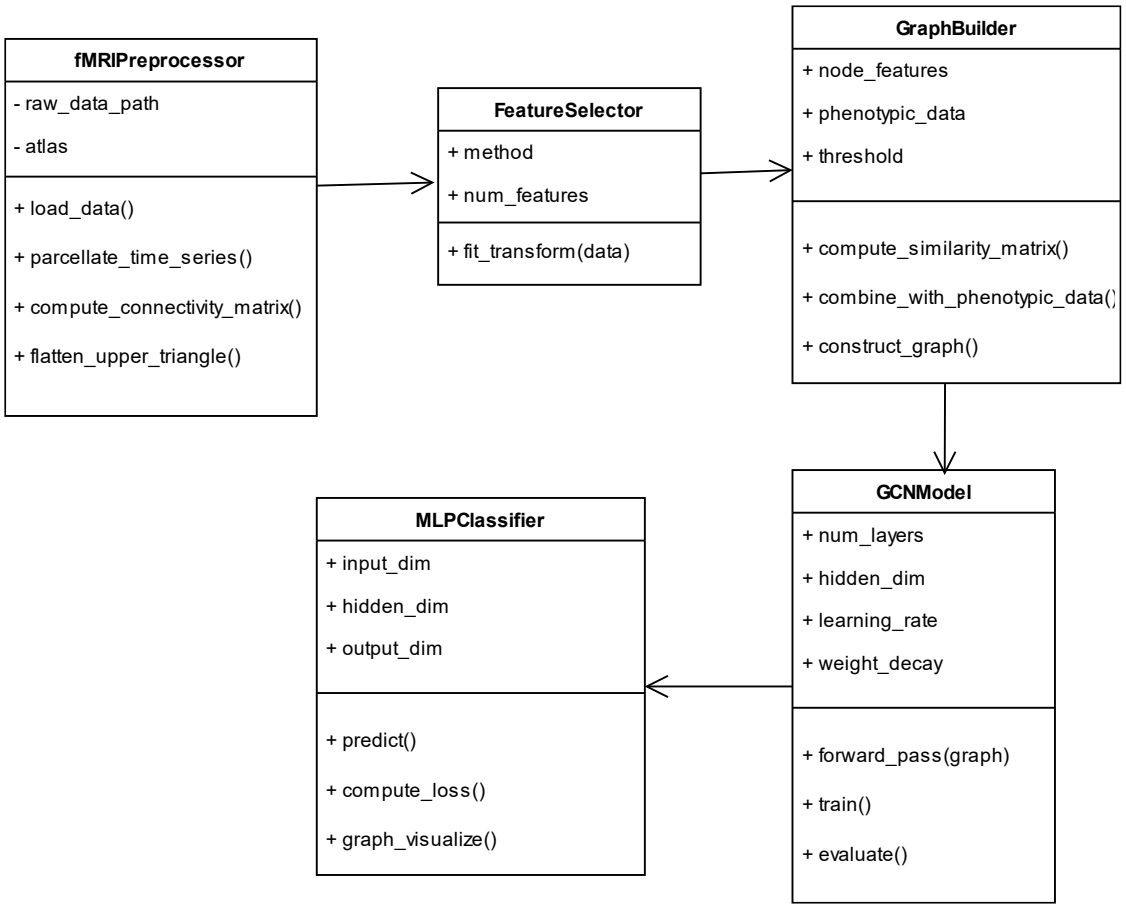

Fig 4.3: Class Diagram for ASD Classification system

The UML class diagram models the core components of the proposed ASD classification system using graph neural networks. It defines the interaction between five key classes:

a. **fMRIPreprocessor:** his class handles the initial data processing pipeline. It converts raw resting-state fMRI data into ROI-wise time series using the Harvard-Oxford atlas. It also computes the Pearson correlation matrices and flattens the upper-triangular parts to create initial feature vectors. Additionally, this class includes preprocessing steps such as temporal filtering, motion correction, and normalization to ensure consistency and minimize noise across subjects.

b. **FeatureSelector:** Implements the RFE-SVM feature selection method. It ranks and removes less informative connectivity features, reducing dimensionality and enhancing model generalizability. The output is a compressed subject-feature matrix. This component ensures that only the most discriminative features contribute to the classification task, which is particularly important when working with high-dimensional neuroimaging data and limited sample sizes.

c. **GraphBuilder:** Constructs the population graph using both feature-based similarities and phenotypic data (e.g., age, gender, site). It computes weighted edges and generates the final adjacency matrix to define graph structure. The graph construction process may use a similarity threshold or k-nearest neighbors approach, and the phenotypic similarity is encoded as additional weights to model inter-subject relationships more accurately.

d. **GCNModel:** Applies graph convolution layers to aggregate information from neighboring nodes in the graph. It learns rich node embeddings that capture both functional connectivity and population-level relationships. This model leverages the message-passing framework of GCNs to iteratively refine node features and can include dropout layers, ReLU activations, and multiple hidden layers to improve expressiveness and prevent overfitting.

e. **MLPClassifier:** A multilayer perceptron that performs the final classification using the learned node embeddings. It outputs the probability of ASD or typical control for each subject. The classifier includes fully connected layers, activation functions, and a SoftMax layer, and it is trained using cross-entropy loss. This component translates learned graph-based representations into actionable predictions.

The solid arrows between classes indicate directional associations, showing the flow of data and control from preprocessing to final prediction.

4.2.2 Use Case Diagram

Below is the use case diagram for the system:

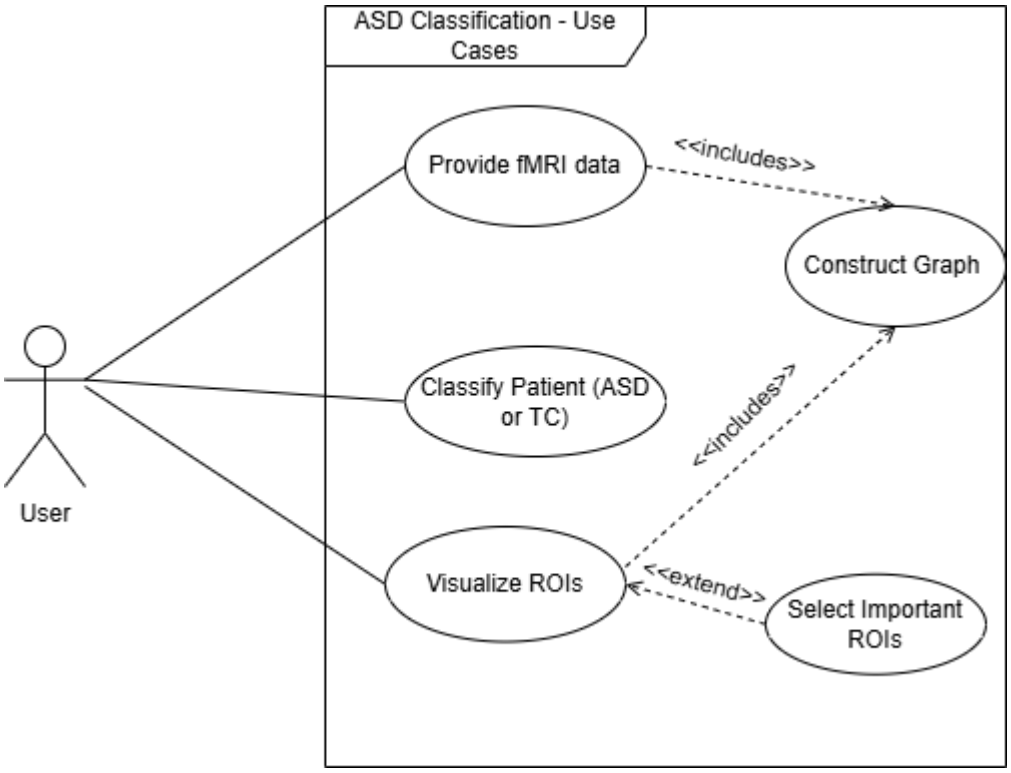

Fig 4.4: Use Case Diagram for ASD Detection System

The use case diagram consists of the following components:

Actor:

1. User

Use Cases:

1. Provide fMRI data
2. Construct graph
3. Classify patients
4. Select important ROIs
5. Visualize ROIs

The use case diagram for the ASD classification system involves a single primary actor—the user, who may be a clinician, researcher, or data analyst. The system is designed to support several key functions to streamline the process of analyzing fMRI data for the purpose of identifying ASD-related brain activity patterns and performing classification.

The first use case is "Provide fMRI data," where the user uploads or inputs preprocessed fMRI data into the system. This data forms the foundation for all downstream tasks. Once the data is provided, the system proceeds to "Construct graph," where a brain graph is generated based on the functional connectivity between different brain regions. In this graph, each node represents a region of interest (ROI), and edges represent the correlation or connectivity strength between pairs of ROIs.

Next, the system enables the "Classify patients" use case. Here, the constructed graph is fed into a Graph Convolutional Network (GCN) model that has been trained to differentiate between ASD and control subjects. Based on the learned patterns in the graph, the model predicts the classification label.

An important aspect of the system is the ability to "Select important ROIs," where the model's interpretability mechanisms identify which brain regions were most influential in making the classification decision. These regions can offer valuable insights into neural markers of ASD.

Finally, the user can "Visualize ROIs" using a built-in 3D brain visualization tool. This interactive module allows the user to explore and interpret the importance scores and spatial distribution of the selected ROIs on the brain, enhancing understanding of the results and facilitating research communication.

Together, these use cases support a complete pipeline for graph-based ASD classification from fMRI data, with strong emphasis on interpretability and user-guided exploration.

4.2.3 Sequence Diagram

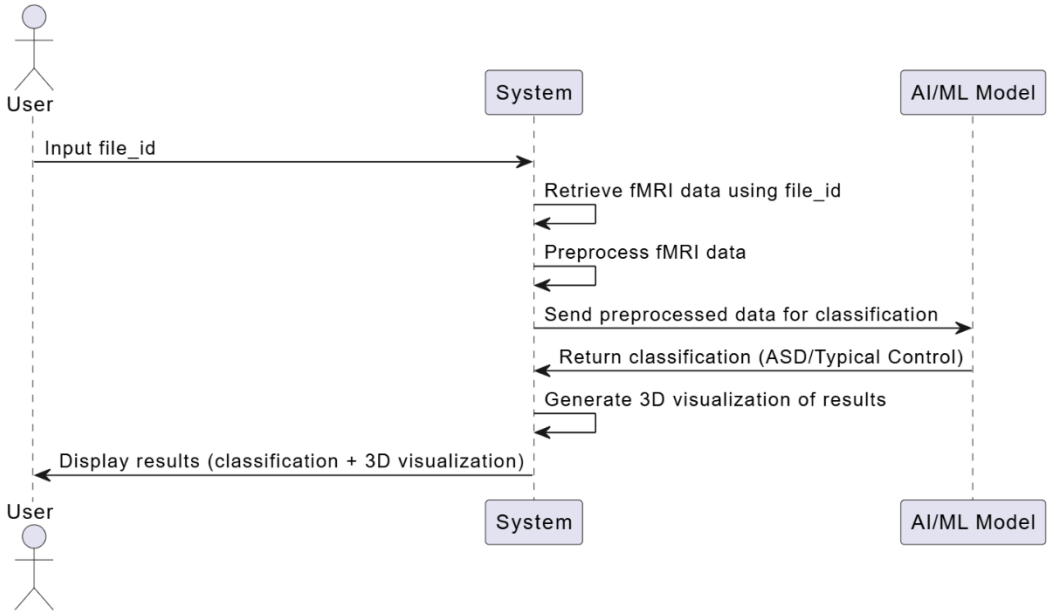

Fig 4.5: Sequence Diagram for ASD Detection

The sequence diagram presented models the interactive workflow of an ASD detection system utilizing resting-state fMRI data. It highlights the sequential interaction between the User, the System, and the AI/ML Model, outlining the end-to-end pipeline from input to result visualization.

1. User Input

   The process begins when the User provides a file_id, corresponding to a specific subject's fMRI scan. This serves as the entry point for the data processing workflow.

2. Data Retrieval and Preprocessing

   Upon receiving the file ID, the System:

   a. Retrieves the relevant raw fMRI data.
   b. Applies preprocessing operations such as motion correction, normalization, and extraction of ROI-wise time series, typically using a standard brain atlas (e.g., Harvard-Oxford).

3. Model Invocation and Classification

   The preprocessed fMRI features are then forwarded to the AI/ML Model, which performs the classification. The model outputs a diagnostic label—either ASD (Autism Spectrum

Disorder) or Typical Control (TC)—based on learned connectivity patterns.

4. Result Processing and Visualization
    a. After receiving the classification result, the System:
    b. Generates an interpretable 3D visualization of the brain regions involved, aiding in transparency and explainability of the AI decision.

5. Display Output to User
   The final results, comprising both the classification outcome and the 3D visualization, are returned to the User. This marks the completion of the analysis cycle.

4.2.4 Activity Diagram

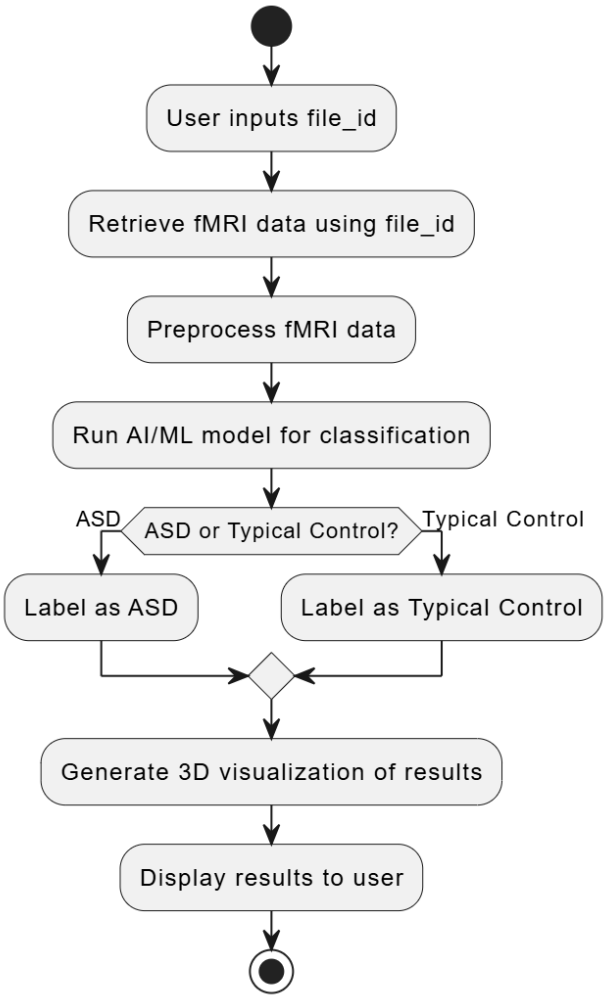

Fig 4.6: Activity Diagram for ASD Detection System

The activity diagram represents the complete operational pipeline of an AI-powered application designed to classify Autism Spectrum Disorder (ASD) from functional Magnetic Resonance Imaging (fMRI) data. This visual workflow illustrates the logical sequence of actions from user input to the final output visualization, ensuring a transparent understanding of the system's functioning. Below is a detailed explanation of each step:

1. **User Inputs file_id**

The system initiates when a user (such as a researcher, clinician, or analyst) inputs a file_id. This identifier corresponds to a subject's fMRI dataset stored securely in the backend. This step ensures individualized access to data while maintaining confidentiality.

2. **Retrieve fMRI Data Using file_id**

Once the file ID is submitted, the application retrieves the corresponding fMRI file from protected storage. Proper validation is performed to ensure the file exists and is authorized for processing, safeguarding against unauthorized access or file manipulation.

3. **Preprocess fMRI Data**

Preprocessing involves several crucial steps to prepare the fMRI data for machine learning input:

1) Motion correction
2) Temporal filtering
3) Parcellation based on a standard brain atlas (HO atlas)
4) Graph construction from time series (nodes = brain regions, edges = functional connectivity)
5) This step transforms the raw data into a structured format suitable for deep learning models.

4. **Run AI/ML Model for Classification**

The preprocessed graph-structured fMRI data is then fed into a trained deep learning model (such as a Graph Neural Network like DeepChebNet). The model, which was trained on labeled ASD and control subjects, processes the input to infer the subject's classification based on learned neural connectivity patterns.

5. Decision: ASD or Typical Control

A decision node checks the model's output to determine whether the subject is classified as having ASD or as a typical control. This decision is made using a softmax/sigmoid threshold on the model's output probability.

6. Label as ASD or Typical Control

Depending on the decision, the subject is labeled as either:

1) ASD (Autism Spectrum Disorder): If the model detects ASD-related patterns.
2) Typical Control: If no ASD-specific patterns are found and the brain activity resembles that of neurotypical individuals.
3) This label is crucial for downstream interpretation and visualization.

7. Generate 3D Visualization of Results

A unique feature of the system is its ability to provide a 3D visualization of brain activity. Using libraries like PyVista or Plotly, the system highlights important brain regions that contributed to the model's decision. This visualization improves interpretability and can aid clinicians or researchers in understanding the biological basis of the prediction.

8. Display Results to User

The final results, including the predicted label (ASD or TC) and the 3D brain visualization, are displayed in a user-friendly web interface. This interface is typically built using Dash (Plotly), allowing for interactive exploration and exportable visuals.

9. End

The process concludes here. The user can choose to input a new file ID or save the results for further analysis.